Laboratory 1

Variant 2, Group 32

By Adam Shahumi and Santiago Puig-Terrero López

1. INTRODUCTION

The goal of this laboratory was to implement a maze solver based on an informed search algorithm and to analyze its behavior under different test scenarios. The maze is represented as a 2D grid where '0' denotes free cells and '1' denotes walls, and the task is to find a path from a given start position to a finish position, or correctly report that no path exists. In this variant, we implemented the Greedy Best-First Search (GBFS) algorithm, using a heuristic to estimate how close a given cell is to the goal.

GBFS is commonly used in real-time pathfinding applications, such as NPC navigation in video games or network routing, where computing a path quickly is more critical than finding the absolute perfect (optimal) route.

- **Advantages**: Its primary advantage is speed and memory efficiency. GBFS always expands the node with the smallest heuristic value, aggressively driving toward the goal without exhaustively searching all paths.

- **Disadvantages**: It does not guarantee an optimal (shortest) path. Furthermore, it can easily get trapped in local minima or dead ends if obstacles are placed directly between the start and the goal.

We experimented with two different heuristics: the Manhattan distance and the Euclidean distance, in order to compare their behavior on grid mazes.

2. IMPLEMENTATION

Our solution is based on the function greedy(maze, start, finish, heuristic_type="manhattan"), which performs the Greedy Best-First Search. We slightly modified the original template by adding the parameter heuristic_type so that the same function can be used with both Manhattan and Euclidean distance. This makes it easier to compare heuristics without duplicating code.

We defined the two heuristics as follows:

```python
def manhattan_heuristic(a, b):
    """Calculates the absolute distance (Manhattan distance) between two points."""
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def euclidean_heuristic(a, b):
    """Calculates the Euclidean distance between two points."""
    return math.sqrt((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2)
```

Manhattan distance measures how many horizontal and vertical steps are needed to reach the goal, while Euclidean distance computes the straight-line distance. On a grid with only four possible movements (up, down, left, right), Manhattan distance matches the real movement constraints more naturally.

The core of the algorithm is the priority queue:

```python
# Priority Queue stores tuples: (heuristic_cost, tie_breaker, current_node, path_taken)
# The tie_breaker ensures Python doesn't crash trying to compare coordinates if costs are equal
counter = 0
pq = []
heapq.heappush(pq, (heuristic(start, finish), counter, start, [start]))

visited = set[Any]()
visited.add(start)
explored_sequence = [] # Keeps track of the exact order nodes were visited
```

Each element in the priority queue is a tuple (heuristic_cost, tie_breaker, current_node, path_taken). We use a counter as a tie-breaker so that Python does not attempt to compare coordinates directly when heuristic values are equal. At every iteration, we pop the node with the smallest heuristic value:

```python
while pq:
    # Pop the node with the lowest heuristic cost
    h_cost, _, current, path = heapq.heappop(pq)
    explored_sequence.append(current)

    # Check if we reached the finish
    if current == finish:
        viz = {
            "maze": maze,
            "start": start,
            "finish": finish,
            "explored": explored_sequence,
            "path": path
        }
        # Number of steps is the length of the path minus the starting node
        return len(path) - 1, viz
```

If the current node is the goal, we return the length of the path (number of steps) and a viz dictionary that contains all information needed for visualization: the maze itself, the start and finish positions, the sequence of explored nodes, and the final path found.

If it is not the goal, we expand all valid neighbors:

```
# Explore valid neighbors
for dr, dc in directions:
    nr, nc = current[0] + dr, current[1] + dc
    neighbor = (nr, nc)

    # Check boundaries and ensure it's not a wall or already visited
    if 0 <= nr < rows and 0 <= nc < cols and maze[nr][nc] == 0 and neighbor not in visited:
        visited.add(neighbor)
        counter += 1

        # Create a new path for this neighbor
        new_path = list[Any](path)
        new_path.append(neighbor)

        # Add to priority queue
        heapq.heappush(pq, (heuristic(neighbor, finish), counter, neighbor, new_path))
```

Neighbors are only added if they are inside the grid, not walls, and not visited yet. The heuristic value for each neighbor determines its priority in the queue, so the algorithm always continues in the direction that looks closest to the goal according to the chosen heuristic.

For visualization, we implemented the function vizualize(viz). It replays the search step by step, printing at each step the current node and the cells that have already been explored:

```
for step, node in enumerate[Any](explored):
    current_explored.add(node)
    print(f"Step {step + 1}: Visiting {node}")

    for r in range(rows):
        row_str = ""
        for c in range(cols):
            pos = (r, c)
            if pos == start:
                row_str += "S "
            elif pos == finish:
                row_str += "F "
            elif pos == node:
                row_str += "X " # Current node being evaluated
            elif pos in current_explored:
                row_str += ". " # Previously explored
            elif maze[r][c] == 1:
                row_str += "+ " # Wall
            else:
                row_str += "  " # Unexplored space
        print(row_str)
```

After all explored nodes are displayed, if a path was found, we print a final map where the solution is marked using * from start to finish. This satisfies the requirement of

visualizing not only the final path but also all cells visited during the search. In the report we included screenshots of these console visualizations for different test scenarios to illustrate the algorithm's behavior. Here are some examples of how the visualization works:

```
Step 1: Visiting (4, 0)
      + F
  +   +
  +   +
  +   +
S +
----------
Step 2: Visiting (3, 0)
      + F
  +   +
  +   +
X +   +
S +
----------
Step 3: Visiting (2, 0)
      + F
  +   +
X +   +
. +   +
S +
----------
```

```
--- Final Solution Path ---

* * * + F
* + * + *
* + * + *
* + * + *
S + * * *

Path from (4, 0) to (0, 4) using greedy best-first search with manhattan heuristic is 16 steps.
```

S - starting node

F - goal/final node

+ - wall (impassable)

X - currently visited node

. - already visited node

* - node that belongs to the final path for the solution

3. DISCUSSION

3.1. Heuristic comparison: Manhattan vs Euclidean

In our implementation, we can switch between Manhattan and Euclidean heuristics by changing the parameter heuristic_type in the scenario function. For simple mazes (like maze1) both heuristics produce a valid path and explore a similar number of nodes. However, there are important conceptual differences:

- Manhattan distance is well aligned with the 4-directional movement in the grid, so it provides a realistic estimate of how many steps are needed to reach the goal.
- Euclidean distance measures the straight-line distance, which is shorter than or equal to the true grid distance. In practice, this means it may underestimate the cost and consider some nodes as equally "good" even if they require different numbers of horizontal and vertical moves.

In mazes where the path is relatively direct, both heuristics behave similarly. In more constrained layouts with obstacles, Manhattan tends to guide the search in a more structured "orthogonal" way, while Euclidean sometimes leads to slightly different exploration patterns.

3.2. Test cases and special scenarios

We designed six different test cases to highlight both the strengths and weaknesses of Greedy Best-First Search and to compare the Manhattan and Euclidean heuristics in practice. Overall, these mazes cover solvable and unsolvable instances, simple geometries and more complex shapes, and situations where the heuristic behaves well or becomes misleading. For testing the template 5x5 maze matrix was used, with a slight change to a 5x6 maze in the last test case to do a simple check if the code/algorithm also work with other maze sizes.

❖ Test case 1 – Maze 1 (path found to goal)

This maze has a clear path from the start in the top-left to the finish in the bottom-right. GBFS quickly finds a valid path using both Manhattan and Euclidean heuristics, and the number of explored nodes is very similar in both cases.

```
Step 17: Visiting (4, 4)
S + . . .
. + . + .
. . . + .
+ . +   .
. . . + F
----------

--- Final Solution Path ---

S + * * *
* + * + *
* * * + *
+   +   *
      + F

Path from (0, 0) to (4, 4) using greedy best-first search with manhattan heuristic is 12 steps.
```

This test confirms that the basic implementation is correct and that the algorithm is efficient in a relatively simple environment where the goal is roughly "diagonal" from the start. In this particular test case we can also see that the algorithm also attempted a different path initially, but met a dead end after which it tried a different path and found the goal node.

❖ Test case 2 - Maze 2 (no path to goal)

In this maze the finish is completely blocked by walls. The algorithm explores all reachable free cells and finally returns -1 as the number of steps, indicating that there is no valid path. Both Manhattan and Euclidean variants reach the same conclusion: the instance is unsolvable, although the exact visiting order may differ slightly. This scenario shows that our implementation correctly handles impossible mazes and does not get stuck in an infinite loop.

```
Step 14: Visiting (1, 4)
S + . . .
. + . + X
. . . +
+ . +   +
. . . + F
----------
Step 15: Visiting (2, 4)
S + . . .
. + . + .
. . . + X
+ . +   +
. . . + F
----------

No valid path was found.

No path from (0, 0) to (4, 4) exists with manhattan heuristic.
```

❖ Test case 3 - Maze 3 (S-shaped maze)

This maze forces the search to follow a long, winding corridor shaped roughly like an “S”. Although the goal is not very far in terms of Euclidean distance, the algorithm has to follow the corridor step by step. With Manhattan distance, the heuristic perfectly matches the true number of horizontal and vertical moves, so the exploration order is very regular along the corridor, while Euclidean sometimes considers several cells equally close in straight-line distance. In practice, both heuristics still find a path and remain relatively fast, but this maze illustrates how the search is strongly guided by the corridor structure rather than by “as-the-crow-flies” distance.

```
--- Final Solution Path ---

* * * + F
* + * + *
* + * + *
* + * + *
S + * * *

Path from (4, 0) to (0, 4) using greedy best-first search with manhattan heuristic is 16 steps.
```

❖ Test case 4 - Maze 4 (C-shaped maze, wall between start and finish)

Maze 4 features a vertical wall separating the start (0,0) and finish (0,4), creating a "C-shaped" trap. This test perfectly highlights a fundamental weakness of GBFS: its vulnerability to direct obstacles. Interestingly, the Euclidean heuristic produced a 14-step path, while the Manhattan heuristic found an optimal 12-step path.

- ❖ **The Euclidean Trap (14 steps):** Because Euclidean measures straight-line distance, it assumes the inner cells of the "C" shape are strictly closer to the goal. The algorithm aggressively walks into the dead-end. Since GBFS locks in its path history as it explores, this inefficient detour is permanently kept in the final path.

```
--- Final Solution Path ---

S * +   F
  * +   *
* * +   *
* + + + *
* * * * *

Path from (0, 0) to (0, 4) using greedy best-first search with euclidean heuristic is 14 steps.
```

- ❖ **The Manhattan Tie-Breaker (12 steps):** Manhattan distance calculates strictly vertical and horizontal steps. At the critical branching point, the Manhattan heuristic assigns the exact same estimated cost (5) to both the inner dead-end node and the correct outer-corridor node. Because our priority queue uses an insertion counter as a tie-breaker, it resolves this tie by prioritizing the older node (the outer corridor), bypassing the trap entirely.

```
--- Final Solution Path ---

S   +   F
*   +   *
*   +   *
* + + + *
* * * * *

Path from (0, 0) to (0, 4) using greedy best-first search with manhattan heuristic is 12 steps.
```

The takeaway in this scenario is that it demonstrates that GBFS does not inherently guarantee the shortest path; it simply takes the first route that satisfies its immediate heuristic calculations. The Manhattan heuristic only found the optimal path here thanks to favorable grid-math aligning perfectly with our queue's chronological tie-breaker.

❖ Test case 5 - Maze 4 (start and goal nodes are the same)

This edge case tests the algorithm's behavior when the starting position is exactly the same as the target destination. For both the Manhattan and Euclidean heuristics, the estimated cost to the goal is immediately calculated as 0. During the very first iteration, the algorithm pops the start node from the priority queue, recognizes it as the finish line, and halts execution before expanding any neighboring cells. This scenario confirms that the fundamental base condition of our implementation functions correctly. It efficiently returns a 0-step path without performing any redundant neighbor calculations or falling into an initialization loop.

```
--- Starting Search Visualization ---

Step 1: Visiting (0, 0)
S   +
    +
    +
  + + +

----------

--- Final Solution Path ---

S   +
    +
    +
  + + +


Path from (0, 0) to (0, 0) using greedy best-first search with manhattan heuristic is 0 steps.
```

❖ Test case 6 - Maze 5 (blocked goal, 2 potential paths)

```
--- Starting Search Visualization ---

Step 1: Visiting (0, 0)
S     + F
  +   + +
  +   +
  +   +
  +
----------
```

```
Step 13: Visiting (2, 4)
S . . + F
. + . + +
. + . + X
  + . + .
  + . . .
----------
Step 14: Visiting (3, 0)
S . . + F
. + . + +
. + . + .
X + . + .
  + . . .
----------
```

```
Step 14: Visiting (3, 0)
S . . + F
. + . + +
. + . + .
X + . + .
  + . . .
----------
Step 15: Visiting (4, 0)
S . . + F
. + . + +
. + . + .
. + . + .
X + . . .
----------

No valid path was found.

No path from (0, 0) to (0, 4) exists with manhattan heuristic.
```

This scenario features a maze where the goal is ultimately blocked, but the starting position branches into two distinct paths. It tests how the algorithm handles dead ends when alternative routes exist.

The algorithm greedily prioritizes the path that heuristically appears more promising, exploring it deeply until it hits a dead end near the finish line (Step 13). Because unvisited nodes from the alternative path were stored in the priority queue earlier, the algorithm seamlessly "jumps" back to explore the second route (Steps 14 and 15). Once this second path also results in a dead end and the queue empties, it correctly identifies the maze as unsolvable.

This visually demonstrates that while GBFS aggressively follows the most promising route first, it does not discard alternative options. It keeps other branches in the priority queue and will reliably backtrack to exhaust all reachable cells before concluding that no valid path exists.

❖ Test case 7 - Maze 6 (5x6 matrix):

This scenario modifies a standard test case by adding an extra column of free space to the right (modified maze5), changing the maze dimensions from the standard 5x5 template to a 5x6 matrix.
The algorithm successfully navigates the rectangular grid, dynamically identifying the new boundaries without throwing any out-of-bounds errors. It accurately calculates the heuristics and finds a valid path from the start to the goal just as it would in a square maze.
This proves that our implementation is flexible and not hardcoded to the default 5x5 matrix size. It demonstrates that the algorithm dynamically handles row and column limits, making it scalable to rectangular grids of varying dimensions and aspect ratios.

```
--- Final Solution Path ---

S * * + F *
  + * + + *
  + * + * *
  + * + *
  + * * *

Path from (0, 0) to (0, 4) using greedy best-first search with euclidean heuristic is 14 steps.
```

4. CONCLUSION

➢ What we learned

In this laboratory, we successfully implemented a Greedy Best-First Search (GBFS) algorithm to solve grid-based mazes. By comparing the Manhattan and Euclidean heuristics, we learned firsthand how the choice of heuristic dictates the order of node expansion and directly impacts the final path. We observed that while GBFS is highly efficient in open environments, it does not guarantee the globally shortest path. Specifically, our test cases revealed how the algorithm can be easily misled by obstacles placed directly between the start and goal (the "C-shaped" trap), and how priority queue tie-breaking mechanisms can drastically alter the final route. We also saw that the algorithm robustly handles dead ends by falling back on previously queued alternative routes, and adapts seamlessly to non-square grids.

➢ What was the challenge

From a practical point of view, the most challenging aspect was correctly managing the priority queue and keeping track of the visited set to prevent infinite loops, while still allowing the algorithm to jump between different branches when hitting a dead end. Designing meaningful test cases that explicitly triggered the underlying weaknesses of the algorithm—such as exposing the Euclidean distance trap—required careful

planning. Finally, formatting the step-by-step console visualization to clearly display the chronological order of explored cells without cluttering the screen was surprisingly complex.

- **What can be improved**

In future iterations, the most significant improvement would be adding a cumulative path-cost function, effectively upgrading the algorithm from GBFS to A* Search. This would combine the current heuristic estimates with the actual distance traveled, guaranteeing an optimal solution. Additionally, we could parameterize the movement model to allow for diagonal moves (which would make the Euclidean heuristic much more applicable) and test the algorithm on substantially larger, procedurally generated mazes to better analyze its time complexity and scalability limits.